

Rapport parallélisme

Nom: Bastien NGUYEN

Numéro étudiant: 22300553

Groupe: B11

Exercice 2



UNIVERSITÉ
TOULOUSE III
PAUL SABATIER



Université
de Toulouse

Introduction

Pour réaliser cet exercice j'ai d'abord vérifié combien de cœur j'avais sur ma machine: 14 pour les machines de l'université et 6 pour la mienne. Tous les résultats que je montre seront faits à partir de ma machine.

Mon archive contient 4 fichiers .c répondant aux questions qui sont:

- ex2-seq pour la version séquentielle
- ex2-par1 pour la première version parallèle
- ex2-par2 pour la seconde version parallèle
- ex2-par3 pour la troisième version parallèle

Et un autre fichier ex2-moy qui prend toutes les versions et les compare en effectuant une moyenne de plusieurs de plusieurs exécutions. Je vais utiliser ce programme pour mes résultats.

J'ai paramétré ce dernier programme de manière à pouvoir choisir jusqu'à combien de thread on va tester les versions. Par exemple si on met 6 alors mon programme va tester avec 1,2,3,4,5 et 6 thread pour les versions parallèles. De plus j'ai rajouté aussi un paramètre pour choisir le nombre d'exécution utilisé pour effectuer la moyenne. Si on choisit 3 alors 3 exécutions seront faites pour calculer une moyenne. Cela me permet d'avoir des résultats plus sûrs mais bien évidemment cela prend plus de temps car il faut calculer bien plus d'exécution.

Exemple d'exécution

```
splat31@DESKTOP-NOKH2P4:~/parallelisme/ex2$ gcc -fopenmp ex2-moy.c -o ex2-moy
splat31@DESKTOP-NOKH2P4:~/parallelisme/ex2$ ./ex2-moy 12 10
```

Temps d'exécution pour la version séquentielle
-séquentielle: 0.378798

Temps d'exécution des versions parallèles avec 1 threads:
-parallèle version 1: 0.612721
-parallèle version 2: 0.398727
-parallèle version 3: 0.391908

Temps d'exécution des versions parallèles avec 2 threads:
-parallèle version 1: 0.382522
-parallèle version 2: 0.255321
-parallèle version 3: 0.252748

Temps d'exécution des versions parallèles avec 3 threads:
-parallèle version 1: 0.285432
-parallèle version 2: 0.199236
-parallèle version 3: 0.193798

Temps d'exécution des versions parallèles avec 4 threads:
-parallèle version 1: 0.212270
-parallèle version 2: 0.151696
-parallèle version 3: 0.151546

Temps d'exécution des versions parallèles avec 5 threads:
-parallèle version 1: 0.187510
-parallèle version 2: 0.128002
-parallèle version 3: 0.143627

Temps d'exécution des versions parallèles avec 6 threads:
-parallèle version 1: 0.172160
-parallèle version 2: 0.127096
-parallèle version 3: 0.122479

Temps d'exécution des versions parallèles avec 7 threads:
-parallèle version 1: 0.158058
-parallèle version 2: 0.109777
-parallèle version 3: 0.111510

Temps d'exécution des versions parallèles avec 8 threads:
-parallèle version 1: 0.145526
-parallèle version 2: 0.113403
-parallèle version 3: 0.109108

Temps d'exécution des versions parallèles avec 9 threads:
-parallèle version 1: 0.147853
-parallèle version 2: 0.104848
-parallèle version 3: 0.104648

Temps d'exécution des versions parallèles avec 10 threads:
-parallèle version 1: 0.138995
-parallèle version 2: 0.114476
-parallèle version 3: 0.101361

Temps d'exécution des versions parallèles avec 11 threads:
-parallèle version 1: 0.141310
-parallèle version 2: 0.103616
-parallèle version 3: 0.102079

Temps d'exécution des versions parallèles avec 12 threads:
-parallèle version 1: 0.194505
-parallèle version 2: 0.181209
-parallèle version 3: 0.157415

Séquentielle

Cet version fut très rapide à faire car on choisit de mettre que des 1 dans la matrice. J'ai choisis de mettre 20000 en size pour que le calcul prennent plus de temps. Au final c'est plus l'initialisation de la matrice qui prend plus de temps car c'est de l'écriture alors que calculer la somme revient juste à faire de la lecture.

En moyenne, le temps d'exécution de ce programme est d'environ 0.375 secondes. Comme c'est du séquentielle il n'y as pas vraiment grand chose à remarquer.

Parallèle version 1

Pour réaliser cette version j'ai choisi d'utiliser un tableau d'entier "int spartielle[nombre threads];" qui aura pour but de stocker les sommes partielles calculées par les threads.

Les threads sont créés avec un `parallel for` classique et vont stocker chacun leurs sommes partielles dans leurs cases du tableau qu'ils connaissent grâce à leur numéro de thread.

Après ce calcul, un seul thread s'occupera d'ajouter toutes les sommes partielles à la vraie somme. J'utilise un `single` il faut donc bien faire attention à ne pas rajouter un `nowait` au `for` car sinon un thread pourrait tenter de calculer la somme finale à partir de calculs non finis de sommes partielles.

Lors de mes tests j'ai remarqué que cette version était très longue (de l'ordre de 5 secondes) et surtout que plus il y avait de threads moins cela était bien ce qui m'a extrêmement surpris et semble incohérent.

Après quelques recherches (ChatGPT je vais pas mentir), j'ai compris qu'en fait cela était dû à mon tableau de sommes partielles. Un phénomène appelé "Padding" arrivait sur ce tableau. En effet du à comment sont faits les caches mémoires les threads se gênent entre eux pour accéder au tableau. Ils utilisent la même ligne du cache ce qui va considérablement ralentir la mémoire.

La solution est donc de faire en sorte qu'ils n'utilisent pas la même ligne du cache.

Parallèle version 1

Pour régler le problème du padding J'ai donc changer la définition du tableau.

Je suis passé de:

-int spartielle[nombre threads];

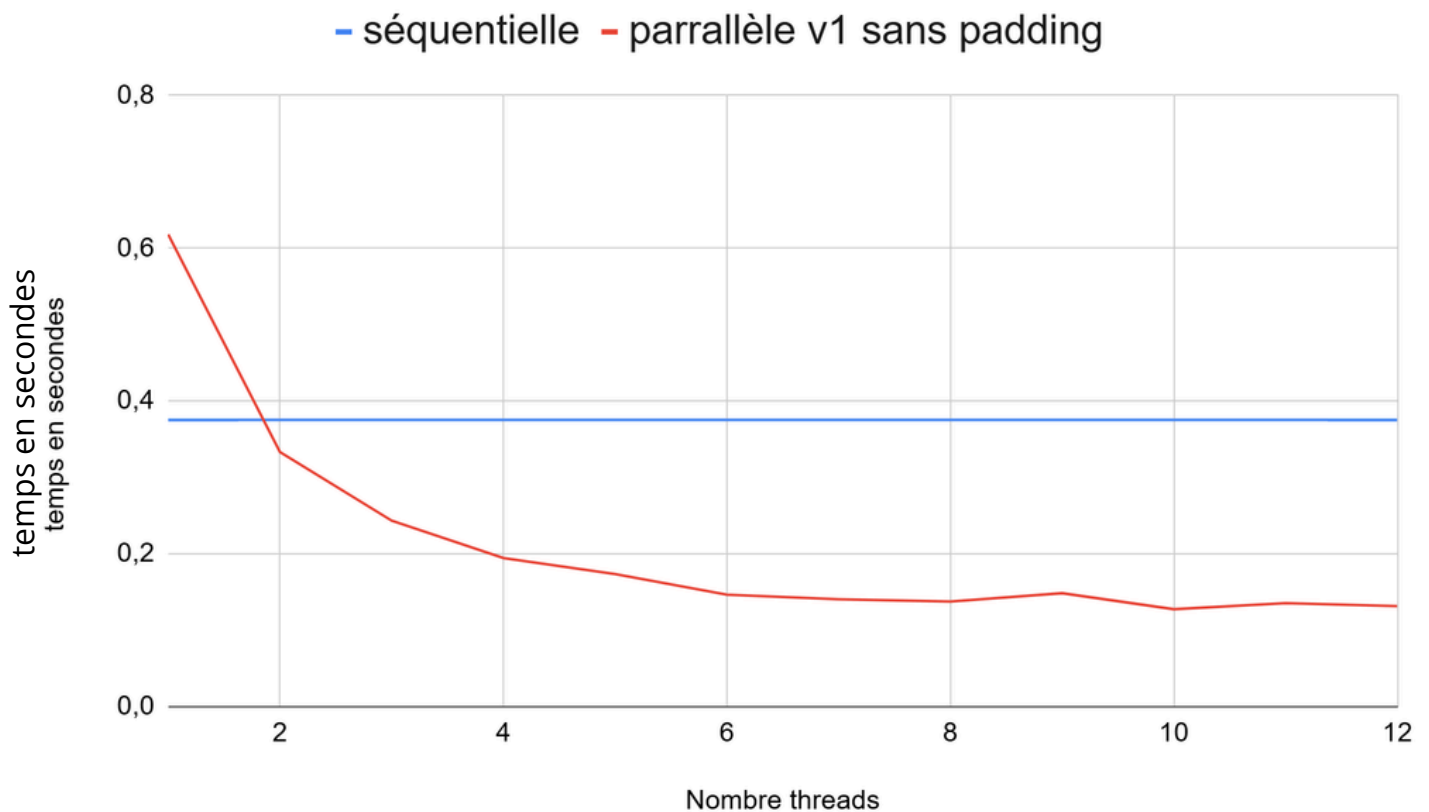
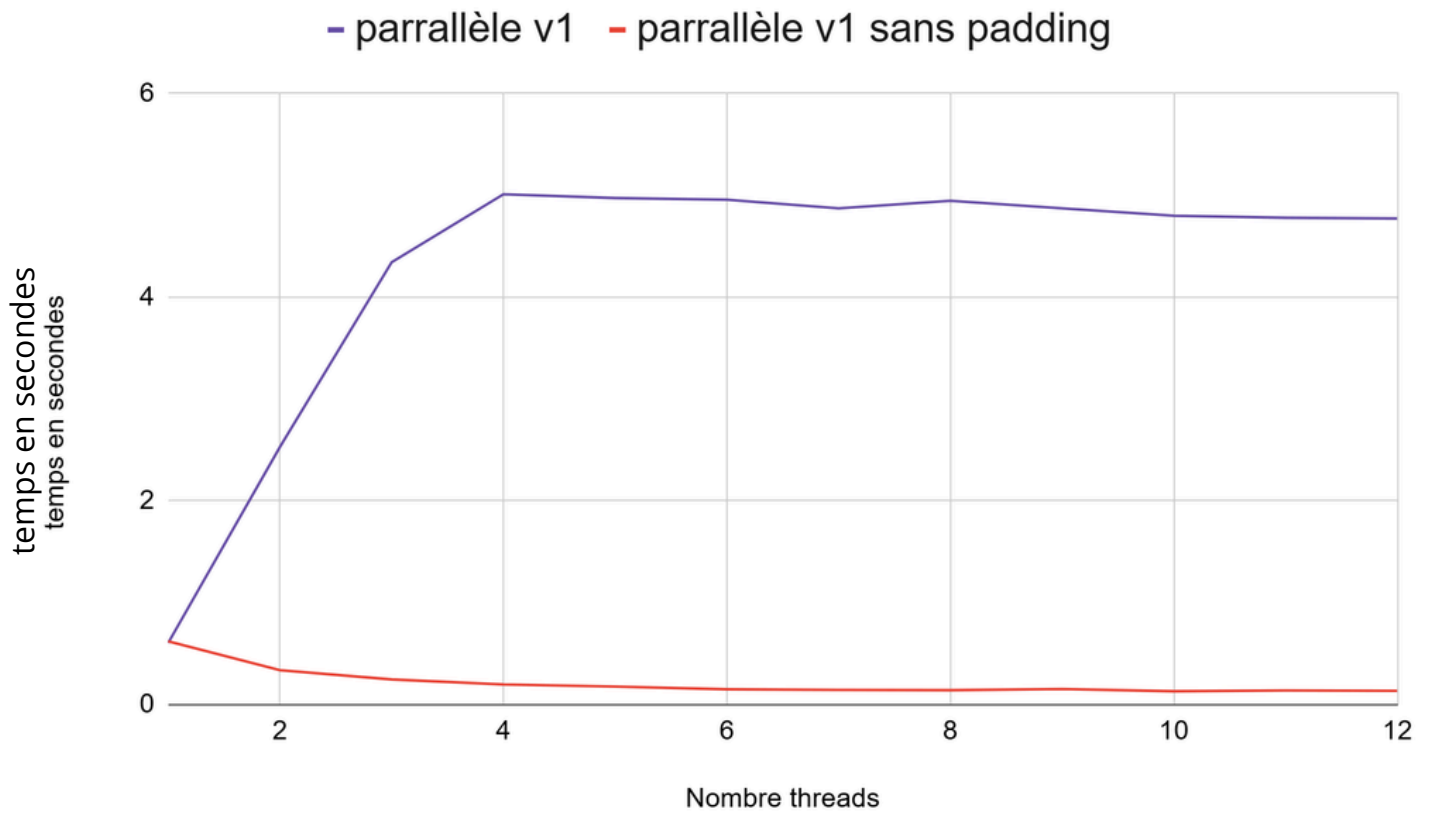
à

-int spartielle[nombre threads][16];

Cela force les threads à occuper une ligne du caches car une ligne de caches étant composé de 64 octets et qu'un entier prend 4 octets ($16 \times 4 = 64$) alors le tableau sera stocker de tel sorte à ce que chaque éléments prennent une ligne du caches. Pour la lecture et l'écriture de sommes partielles on ne se servira que de `spartielle[nb][0]` et on ignorera les cases qu'on as créés (1 à 15). C'est un cas ou pour optimiser la vitesse on va choisir d'utiliser plus de mémoire.

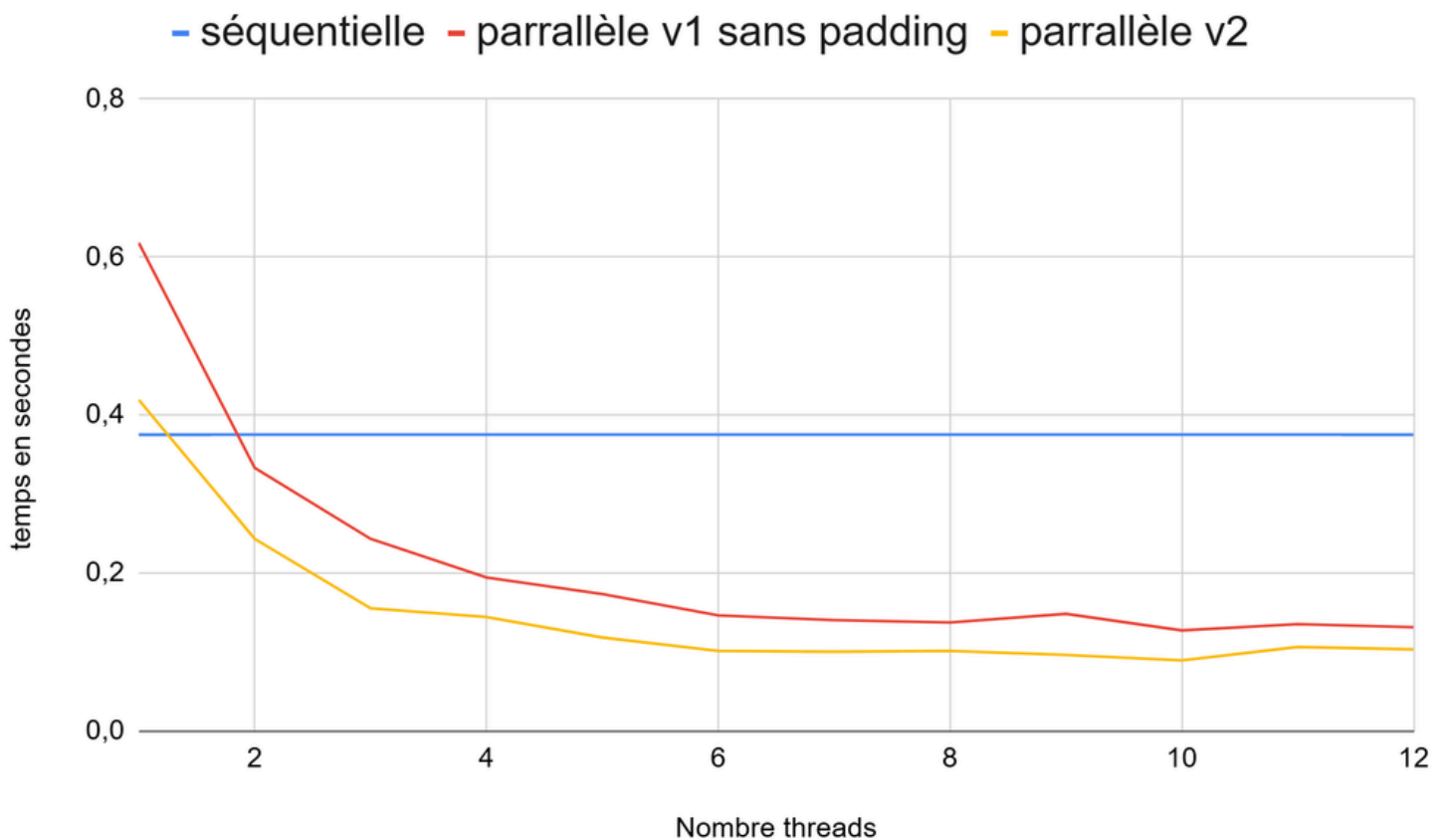
Les observations temporelles (pages suivantes) nous montre bien que le parrallèle est bien mieux que le séquentielle sauf pour 1 car cela reviens à faire du séquentielle avec des primitives inutiles.

Parallèle version 1



Parallèle version 2

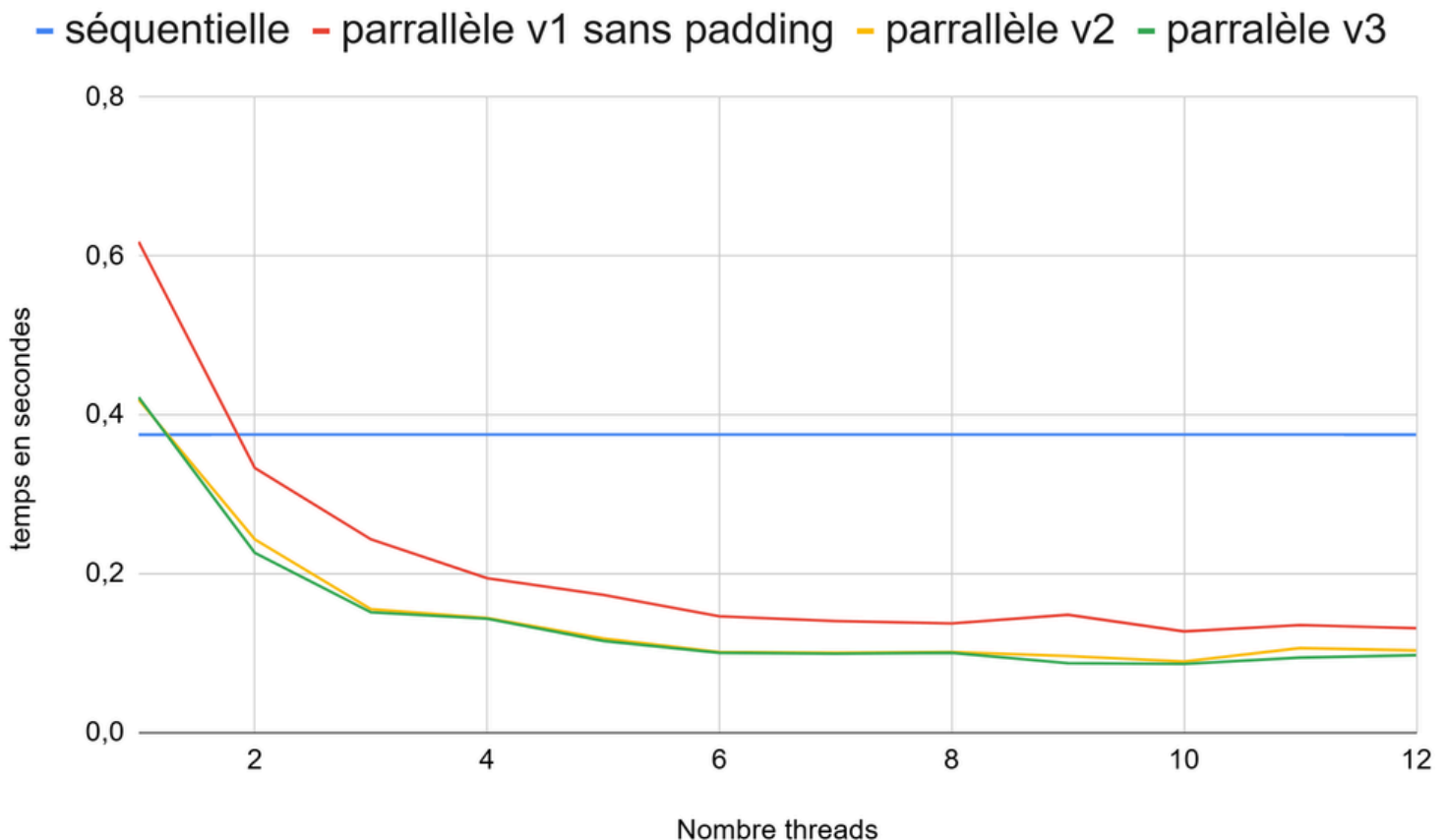
Pour cette version, chaque thread possède en private leur propre somme partielle "spartielle" et s'occupe de calculer celle-ci. Enfin dès qu'ils ont calculer leurs sommes partielles ils vont rentrer en section critique pour la rajouter à la somme final. Cela est bien plus efficace que la version1 car on utilise une variable private et comme le atomic est effectué qu'une seule fois on perd moins de temps en étant en section critique qu'en accédant plein de fois à des cases d'un tableau.



Les observations temporelles prouvent bien que cette version fait mieux que la version 1 et on observe bien que l'accélération stagne quand on dépasse le nombre de coeur de la machine.

Parallèle version 3

Pour cette version, j'utilise la primitive "reduction (+:somme)" pour encore plus optimiser le programme. Cette version est la plus la plus rapide bien que vu que les calculs soient simples elles soient similaires en temps à la version 2 voir même pires si on a pas de chance. Elle reste la version à privilégier si on veut paralléliser ce programme.



permet pas encore de bien observer l'efficacité de la réduction mais on remarque cependant que pour 1 thread on ne perd pas beaucoup de temps pour exécuter les primitives et puis qu'à partir de 2 threads on remarque bien l'intérêt de paralléliser un programme.